

Assignment 2 Report

The Professional Master Of
Embedded And Cyber-physical Systems



ECPS 216 - F'24: IOT Systems & Software

Under the guidance of

Prof. Quoc-Viet Dang

Name of Candidates

Sahil Kakadiya

UCI Student ID

93282494

The given code is an implementation of a Wi-Fi enabled ESP8266-based system that communicates with a target device (Raspberry Pi) over UDP protocol. It reads data from an LDR (Light Dependent Resistor) sensor, processes the data to compute averages, and sends this information to the target device. Additionally, the system supports state-based transitions, including handshake synchronization with the target device and continuous data transmission.

Here is the code flow chart.

1. Code Flow in ESP32

1.1 Setup Function

- Initializes GPIO pins and establishes Wi-Fi connectivity.
- Once Wi-Fi is connected, it prints the device's IP address.
- Initializes UDP communication on the defined listening port (54321).

1.2 Loop Function

- The system checks its current state (IDEL, HANDSHAKE_DONE, SENDING_DATA) and performs actions accordingly.
- In the **IDEL** state, it waits for a configuration message (handshake) from the Raspberry Pi. Which contains the Device ID token to check that the Connection wants to establish is actually a Raspberry PI.
- After the handshake, the system configures timers and starts sending data.
- In the **SENDING_DATA** state, it continues data transmission and checks for stop command.

1.3 Interrupt-Based Actions (Using Ticker)

The Ticker library provides software-based timers. Three timers are used:

- **ledTicker**: Toggles the LED at regular intervals (0.5 seconds).
- **ldrTicker**: Reads sensor data every second.
- **udpTicker**: Sends the average sensor data over UDP every two seconds.

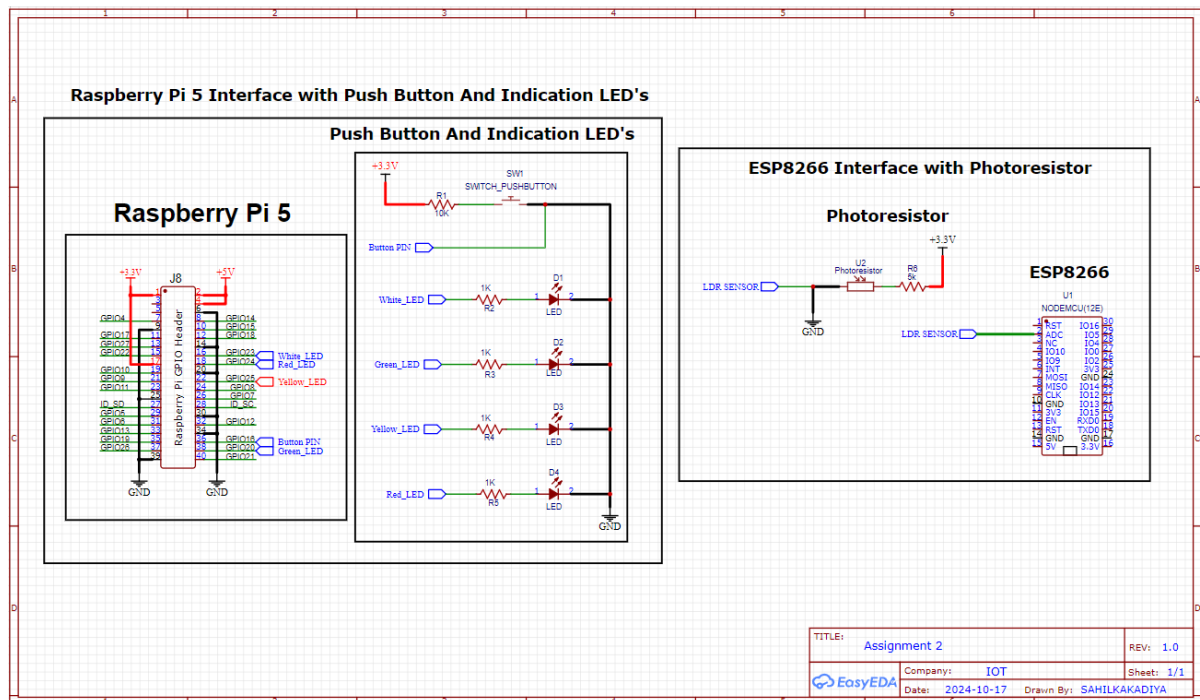
Note: Using the Ticker library it solves the delay() problems and ticks the software to the run the specific code at the right time.

2. Code Flow in Raspberry Pi code

This Python script automates communication between a Raspberry Pi and an ESP8266 device via UDP, controlling LEDs based on sensor data. It follows these key steps:

1. **Wi-Fi Connectivity:** It checks and reconnects to Wi-Fi if needed.
2. **UDP Socket Setup:** It initializes a socket to send/receive messages with the ESP8266.
3. **IP Reachability:** It pings the ESP8266 to ensure it's reachable.
4. **Handshake:** A handshake process is initiated to start communication with the ESP8266.
5. **Sensor Data Handling:** It continuously receives sensor data from the ESP8266 and controls LEDs based on the sensor's intensity levels.
6. **LED Control:** LEDs (Red, Yellow, Green, White) indicate different data intensities or states, and a button toggles program start/stop.

3. SCHEMATIC



This schematic shows two interfaces: one between the **Raspberry Pi 5** and push buttons/LED indicators, and another between the **ESP8266** and a photoresistor (LDR sensor).

Here's an explanation of the different parts:

1. Raspberry Pi 5 Interface with Push Button and LED Indicators

- **GPIO Pins:**
 - The Raspberry Pi's GPIO header (J8) is shown with connections to various GPIO pins. GPIO pins are used to control or receive signals from external components such as LEDs or push buttons.
- **Push Button:**
 - The push button (SW1) is connected to the Raspberry Pi. One side of the button connects to **GPIO Pin** via a 10kΩ resistor (R1) and the other side connects to **3.3V**.
- **LED Indicators:**
 - There are four LEDs connected to different GPIO pins:
 1. **White LED** is connected through a 1kΩ resistor (R2) to a GPIO pin.

2. **Green LED** is connected via a $1k\Omega$ resistor (R3).
3. **Yellow LED** is connected via a $1k\Omega$ resistor (R4).
4. **Red LED** is connected via a $1k\Omega$ resistor (R5).

- **Power and Ground:**

- The Raspberry Pi 5 uses **3.3V** and **5V** power supplies, connected as shown.
- The LEDs and the button are grounded to the GND pin of the Raspberry Pi.

2. ESP8266 Interface with Photoresistor (LDR Sensor)

- **LDR (Photoresistor) Sensor:**

- The light-dependent resistor (LDR) is used to measure light intensity. It changes its resistance based on the amount of light that falls on it.
- One side of the LDR (U2) is connected to **+3.3V**, while the other side is connected to **GND** through a $5k\Omega$ resistor (R6). This creates a voltage divider circuit.
- The only point between the LDR and resistor is connected to the **ADC (Analog-to-Digital Converter)** pin of the **ESP8266**, which can measure the voltage and interpret it as a light level. There is an only ADC Pin A0 which is available in esp8266.